

FightTumor: Scribble-to-Point Adaptation with a Two-Checkpoint DKFZ Ensemble for autoPET V

Baidu Li¹ and Jun Ma^{1,2}

¹ University Health Network, Toronto, ON, Canada

² Princess Margaret Cancer Centre and UHN AI Hub, University Health Network,
Toronto, ON, Canada

Abstract. This technical description documents the final FightTumor autoPET V inference container. The challenge provides corrective scribbles, while the reused DKFZ interactive model expects foreground and background point prompts. A deterministic adapter translates the scribble record into the DKFZ multiple-point JSON contract. The container then applies two unmodified public DKFZ checkpoint files from `fold_0` and `fold_5`, and averages their logits equally. Each checkpoint retains the inherited three-axis mirroring test-time augmentation (TTA): identity plus seven spatial flips per sliding-window tile. No model was trained, fine-tuned, calibrated, or post-processed by FightTumor. We report the deployed prompt bridge and inference configuration, not a new segmentation model or a challenge-performance claim.

Keywords: autoPET V · interactive segmentation · scribbles · point prompts · test-time augmentation

1 Introduction

autoPET V evaluates iterative PET/CT lesion segmentation using corrective scribbles. The selected upstream DKFZ inference workflow expects structured foreground and background points rather than scribbles. This representation boundary must be explicit: otherwise the prompt supplied to an unchanged checkpoint can silently differ from the intended interaction.

FightTumor therefore provides a deterministic scribble-to-point bridge and deploys a compact two-checkpoint DKFZ ensemble. The final Docker command uses `-folds 0,5`. All other downstream inference semantics are retained: native point Euclidean-distance-transform (point-EDT) prompt encoding, sliding-window overlap, Gaussian weighting, mirroring TTA, and the inherited segmentation conversion.

2 Methods

2.1 Prompt adaptation

The adapter accepts either objects in a `points` field or separate `tumor` and `background` arrays. Labels `tumor`, `foreground`, `fg`, `positive`, and `pos` map to

foreground. Labels `background`, `bg`, `negative`, and `neg` map to background. Unrecognized labels do not create prompts.

Every retained coordinate has exactly three numeric values and is rounded to integer voxel coordinates. The output uses the DKFZ `Multiple points` JSON schema with canonical `tumor` and `background` names. Foreground entries precede background entries while retaining input order within each polarity. Both point caps are zero in the final container, so all supplied foreground and background points are retained.

2.2 Inference configuration

PET and CT MHA inputs are read with the inherited DKFZ image reader and stacked as image channels. The adapted points are encoded by the native point-EDT routine. The final container loads only `fold_0/checkpoint_final.pth` and `fold_5/checkpoint_final.pth` from the public DKFZ autoPET IV interactive workflow [2, 3].

For each fold, the inherited 3D nnU-Net inference uses a $192 \times 192 \times 192$ patch, sliding-window step size 0.5, Gaussian overlap weighting, half-precision image input, and mirroring on axes (0, 1, 2). Mirroring evaluates eight transforms per tile: the identity and all seven non-empty flip combinations. Fold-level pre-softmax logits are accumulated with equal weights and divided by two before final segmentation conversion. No tracer-specific SUV filter is enabled.

2.3 Container interface and reproducibility

The image reads CT and PET MHA files from the Grand Challenge input tree, reads its interaction record from `lesion-clicks.json`, and writes one tumor-lesion segmentation MHA file to the required output tree. Because the input mount may be read-only, the entrypoint stages the input tree to a writable working directory before writing the adapted JSON. This staging does not alter CT or PET intensities, geometry, prompt content, or output location.

The final Docker image has tag suffix `2fold_tta_20260830b`. Its exported archive SHA-256 is shown below in four contiguous groups for readable line wrapping:

```
5201c22c4276e4df 357ec12a2cf61315
b9433a2fa94102a6 732509e41ca2fdc5
```

The build receipt locks the folds, point-EDT configuration, and Docker command arguments.

2.4 Training and evaluation scope

FightTumor performed no additional training, fine-tuning, calibration, or model-selection experiment. The reported checks are Docker build, archive, command, and checkpoint-integrity checks. A target-platform GPU runtime test was not completed because the available remote NVIDIA host had a driver/NVML mismatch. Accordingly, no challenge score or target-platform runtime claim is made.

3 Results and limitations

The adapter’s deterministic checks establish label mapping, coordinate rounding, canonical multiple-point JSON output, and uncapped point retention. The final export verifies `-folds 0,5`, the two checkpoint hashes, and archive integrity. These checks establish the software configuration only; they do not estimate segmentation accuracy, robustness, interaction benefit, or runtime on the Grand Challenge platform.

The selected two-checkpoint ensemble is an engineering trade-off intended to reduce inference cost relative to a larger ensemble. It must not be interpreted as a demonstrated accuracy improvement.

4 Conclusion

FightTumor’s final container bridges autoPET V scribbles to the point-prompt interface of an unmodified DKFZ interactive model and averages logits from `fold_0` and `fold_5`. Native point-EDT encoding and three-axis mirroring TTA are retained. No new learned model or performance claim is made.

Acknowledgments. The authors acknowledge the autoPET V organizers and the DKFZ interactive workflow on which the reused checkpoint is based.

Disclosure of Interests. The authors declare no competing interests.

References

1. Medical Image Analysis Group, German Cancer Research Center (DKFZ): autoPETV project repository. <https://github.com/lab-midas/autoPETV> (accessed 2026-08-30)
2. Rokuss, M., Kirchhoff, Y., Isensee, F., Maier-Hein, K.H.: autoPET-interactive. <https://github.com/MIC-DKFZ/autoPET-interactive> (accessed 2026-08-30)
3. Rokuss, M., Kirchhoff, Y., Isensee, F., Maier-Hein, K.H.: Towards interactive lesion segmentation in whole-body PET/CT with promptable models. arXiv:2508.21680 (2025)
4. Bardli: autopetv-scribble-to-point. Private source repository. <https://github.com/Bardli/autopetv-scribble-to-point> (accessed 2026-08-30)

Table 1. Final algorithm details. "Unchanged" denotes an upstream DKFZ component not modified by FightTumor.

Field	Description
Team name	FightTumor
Algorithm name	Scribble-to-point adapter with a two-checkpoint DKFZ ensemble
New component	Deterministic scribble JSON normalization and conversion to multiple-point prompts
Model and architecture	Public DKFZ autoPET IV interactive checkpoint and architecture; unchanged
Checkpoint members	<code>checkpoint_final.pth</code> from <code>fold_0</code> and <code>fold_5</code>
Ensembling	Equal arithmetic mean of the two fold-level logits
Prompt encoding	Native point-EDT; uncapped foreground and background points
Sliding-window inference	192^3 patch, step size 0.5, Gaussian weighting; unchanged
Test-time augmentation	Mirroring on axes (0, 1, 2): identity plus seven flips per tile
Training data and augmentation	None added; no training or fine-tuning by FightTumor
Post-processing	DKFZ segmentation conversion unchanged; tracer SUV filter disabled
Evidence reported	Build, archive, Docker-command, and checkpoint-integrity checks only
Code availability	Adapter source: private repository; public upstream DKFZ code linked below